DOCUMENTATION PACK · 4 OF 6

Self-Hosting & Installation

Docker one-liner, remote TLS exposure, non-Docker PHP, MySQL option, backups, scheduling.

Exposure-Check-OSINT v1.0 · Apache-2.0 · self-hosted · keyless · no AI · https://github.com/RanaKhan77/Exposure-Check-OSINT

Fastest path: `git clone` → `cd Exposure-Check-OSINT` → `./install.sh` → open `http://localhost:8080`. One container, SQLite, zero API keys.

Quick start (Docker, recommended)

```
git clone https://github.com/RanaKhan77/Exposure-Check-OSINT.git
cd Exposure-Check-OSINT
./install.sh            # builds + boots one container, generates your password
```

Open `http://localhost:8080/login` and sign in with the printed password.
That is the whole install. Data (history, settings, license) lives in the `ec-db` and `ec-storage` Docker volumes and survives upgrades.

Custom port: `./install.sh --port 9090`
Custom password: `./install.sh --password 'your-strong-password'`

Daily operations

```
docker compose -f docker/docker-compose.yml logs -f      # follow logs
docker compose -f docker/docker-compose.yml restart      # restart
docker compose -f docker/docker-compose.yml down         # stop (data persists)
docker compose -f docker/docker-compose.yml pull || \
  git pull && docker compose -f docker/docker-compose.yml up -d --build   # upgrade
```

Exposing it remotely (recommended setup)

The container speaks plain HTTP. For anything beyond localhost, put a TLS reverse proxy in front — Caddy makes it one line:

```
exposure.example.com {
    reverse_proxy 127.0.0.1:8080
}
```

nginx equivalent:

```
server {
    listen 443 ssl http2;
    server_name exposure.example.com;
    ssl_certificate     /etc/letsencrypt/live/exposure.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/exposure.example.com/privkey.pem;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Then set `APP_URL=https://exposure.example.com` in `.env` so generated links are correct.

Without Docker (PHP 8.2+ on a VPS)

```
git clone https://github.com/RanaKhan77/Exposure-Check-OSINT.git && cd Exposure-Check-OSINT
composer install --no-dev --optimize-autoloader
cp .env.example .env
php artisan key:generate
sed -i "s|^ADMIN_PASSWORD=.*|ADMIN_PASSWORD=$(openssl rand -hex 12)|" .env
touch database/database.sqlite
php artisan migrate --force
php artisan serve --host 127.0.0.1 --port 8080
```

Requires: PHP 8.2+ with `pdo_sqlite`, `sqlite3`, `curl`, `mbstring`, `dom`, `openssl`.
For a serious non-Docker deployment use php-fpm + your webserver. If you run `php artisan config:cache`, remember to re-run it after ANY `.env` change (cached config stops reading `.env` — the admin password is read at config load time by design).

Using MySQL instead of SQLite

Set in `.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=exposure_check
DB_USERNAME=...
DB_PASSWORD=...
```

Then `php artisan migrate --force`. SQLite is the default because an own-use personal tool should not need a database server.

Backups

Everything that matters is in two places: `storage/app/database.sqlite` (or your MySQL) and `storage/` (which also holds your `.env` copy, logs and logo cache). The Docker volumes map to both. `sqlite3 storage/app/database.sqlite ".backup backup.db"` takes a safe hot copy.

Scheduling (optional, non-Docker)

The license heartbeat/re-validation is scheduled automatically inside Docker via Laravel's scheduler integration. Without Docker, add one cron entry:

```
* * * * * cd /path/to/Exposure-Check-OSINT && php artisan schedule:run >> /dev/null 2>&1
```